

Jenkins Learning Roadmap (with Example Pipelines)

Beginner → Intermediate → Advanced → Expert

Roadmap Checkpoints

Beginner Level (Basics & Foundations)

- Introduction to Jenkins: What is Jenkins, CI/CD concepts, Jenkins architecture.
- Installation on Windows/Linux/Docker and UI Overview.
- Freestyle Jobs: Creating, configuring, SCM integration, build triggers.
- Pipeline Basics: Declarative vs Scripted, basic syntax (pipeline, agent, stages, steps).
- Run a simple Build → Test → Deploy pipeline.

Intermediate Level (Automation & Pipelines)

- Pipeline Features: environment variables, parameters (String, Choice, Boolean, Password, File).
- Post actions: success, failure, always, unstable.
- Control Flow: when, parallel, timeout, retry, input steps.
- SCM & Build Tools: GitHub/GitLab PR pipelines, Maven, Gradle, npm, MSBuild, JUnit test reports.
- Notifications & Reporting: Email, Slack/MS Teams, JUnit & HTML reports.

Advanced Level (Security, Scaling & Integrations)

- Credentials & Secrets: Credentials Manager, withCredentials usage, storing API keys, SSH keys.
- Distributed Builds: Master-Agent setup, labels, Docker agents, Kubernetes dynamic agents.
- Plugins & Integrations: SonarQube, Nexus/Artifactory, Docker, Helm, Kubernetes.
- Security & Access Control: RBAC, securing Jenkins, backup & restore.

Expert Level (CI/CD Mastery & DevOps)

- Advanced Pipelines: Multi-branch, Matrix builds, fast-fail parallel pipelines, Shared Libraries.
- Monitoring & Observability: Prometheus, Grafana, health checks, alerts.
- Enterprise CI/CD: GitOps, Jenkins + Ansible, Jenkins + Terraform, Jenkins + Wazuh for DevSecOps.
- Real-World CI/CD Projects: Build → Test → Scan → Docker → Push → Deploy (Kubernetes/IIS/Nginx).
- Deployment Strategies: Blue-Green & Canary deployments, Cloud integrations (AWS, Azure, GCP).

1) Basic Declarative Pipeline (Build → Test → Archive)

A minimal Jenkinsfile using Declarative syntax. Runs on any agent and executes shell/PowerShell depending on node OS.

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build') {
      steps {
        echo "Building..."
        sh 'echo Build step (Linux/Mac) || true'
        bat 'echo Build step (Windows)'
      }
    }
    stage('Test') {
      steps {
        echo "Running tests..."
        sh 'pytest -q || true'
        junit 'reports/**/*.xml' // publish if exist
      }
    }
    stage('Package & Archive') {
      steps {
        sh 'zip -r artifact.zip . || true'
        archiveArtifacts artifacts: 'artifact.zip', fingerprint: true, onlyIfSuccessful: true
      }
    }
  }
}
```

2) Parameterized Pipeline + env usage

Demonstrates String/Choice/Boolean parameters and safe use of env variables.

```
pipeline {
  agent any
  parameters {
    string(name: 'APP_VERSION', defaultValue: '1.0.0', description: 'App version tag')
    choice(name: 'ENVIRONMENT', choices: ['dev', 'qa', 'prod'], description: 'Target environment')
    booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Execute tests?')
  }
  environment {
    CURR_TS = "${new Date().format('yyyyMMdd_HH:mm:ss')}"
  }
  stages {
    stage('Info') {
      steps {
        echo "Deploying version: ${params.APP_VERSION} to ${params.ENVIRONMENT} at ${env.CURR_TS}"
      }
    }
    stage('Optional Tests') {
      when { expression { return params.RUN_TESTS } }
      steps {
        sh 'pytest -q || true'
        junit allowEmptyResults: true, testResults: 'reports/**/*.xml'
      }
    }
  }
}
```

```

    }
  }
}

```

3) Credentials: withCredentials (Username/Password, Token, SSH key)

Use Jenkins Credentials Manager IDs instead of hardcoding secrets. Avoid Groovy string interpolation that exposes secrets.

```

pipeline {
  agent any
  stages {
    stage('Use Credentials') {
      steps {
        withCredentials([
          usernamePassword(credentialsId: 'dockerhub-creds', usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS'),
          string(credentialsId: 'sonar-token', variable: 'SONAR_TOKEN')
        ]) {
          sh 'echo "Logging into Docker Hub..."'
          sh 'echo "$DH_PASS" | docker login -u "$DH_USER" --password-stdin'
          sh 'sonar-scanner -Dsonar.login=$SONAR_TOKEN || true'
        }
      }
    }
  }
}

```

4) Parallel stages with fast-fail

Run independent test suites in parallel; fail fast if any branch fails.

```

pipeline {
  agent any
  options { parallelsAlwaysFailFast() }
  stages {
    stage('Parallel Tests') {
      parallel {
        stage('Unit') {
          steps { sh 'npm run test:unit' }
        }
        stage('Integration') {
          steps { sh 'npm run test:integration' }
        }
        stage('Lint') {
          steps { sh 'npm run lint' }
        }
      }
    }
  }
}

```

5) Post actions (success, failure, always, unstable)

Use post conditions for notifications and cleanup.

```

pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'make build || true' } }
  }
  post {

```

```

    success { echo 'Build succeeded ■' }
    failure { echo 'Build failed ■' }
    unstable { echo 'Build unstable ■■' }
    always {
        cleanWs()
        echo "Done at ${new Date()}"
    }
}
}
}

```

6) Tools block (Maven, JDK, NodeJS)

Tools must be preconfigured in Manage Jenkins → Global Tool Configuration with these names.

```

pipeline {
    agent any
    tools {
        maven 'maven3'
        jdk 'jdk11'
        nodejs 'node18'
    }
    stages {
        stage('Build with Maven') {
            steps {
                sh 'mvn -v && mvn -B clean package'
            }
        }
        stage('Frontend') {
            steps {
                sh 'node -v && npm ci && npm run build'
            }
        }
    }
}
}

```

7) Multibranch Pipeline: branch-specific behavior

Use the when/branch condition to customize steps per branch.

```

pipeline {
    agent any
    stages {
        stage('Checkout') { steps { checkout scm } }
        stage('Build') {
            when { not { branch 'docs' } }
            steps { sh 'mvn -B clean package' }
        }
        stage('Docs') {
            when { branch 'docs' }
            steps { sh 'make docs' }
        }
    }
}
}

```

8) Shared Library (vars & src)

Reuse common steps. In Jenkins: Manage Jenkins → Global Pipeline Libraries → add library (name: 'acme-lib').

```

@Library('acme-lib@main') _

pipeline {

```

```

agent any
stages {
  stage('Hello') {
    steps {
      sayHello 'Jenkins' // vars/sayHello.groovy
      acmeDockerBuild(image: 'repo/app', tag: '1.2.3') // vars/acmeDockerBuild.groovy
    }
  }
}

```

9) Build & Push Docker Image

Build an image and push to a registry using withCredentials for auth.

```

pipeline {
  agent { label 'docker' }
  environment { IMAGE = 'yourrepo/yourapp' }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build Image') {
      steps {
        sh 'docker build -t $IMAGE:$BUILD_NUMBER .'
      }
    }
    stage('Push Image') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub-creds', usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
          sh 'echo "$DH_PASS" | docker login -u "$DH_USER" --password-stdin'
          sh 'docker push $IMAGE:$BUILD_NUMBER'
        }
      }
    }
  }
}

```

10) Deploy to Kubernetes with Helm

Assumes kubectl/helm available on agent and kubeconfig provided via credentials.

```

pipeline {
  agent { label 'k8s' }
  environment { RELEASE = 'webapp' }
  stages {
    stage('Deploy') {
      steps {
        withCredentials([file(credentialsId: 'kubeconfig-prod', variable: 'KUBECONFIG')]) {
          sh 'helm upgrade --install $RELEASE charts/webapp --set image.tag=${BUILD_NUMBER}'
          sh 'kubectl rollout status deploy/webapp'
        }
      }
    }
  }
}

```

11) Matrix Build (OS x Java)

Compile and test against multiple environments in parallel.

```

pipeline {
  agent none

```

```

stages {
  stage('Matrix') {
    matrix {
      axes {
        axis { name 'OS'; values 'linux', 'windows' }
        axis { name 'JAVA'; values '11', '17' }
      }
      agent { label "${OS}" }
      stages {
        stage('Setup JDK') { steps { echo "Using Java ${JAVA}" } }
        stage('Build') {
          steps {
            sh 'mvn -B -Djava.version=$JAVA clean package || true'
          }
        }
      }
    }
  }
}

```

12) Infra as Code: Terraform + Ansible

End-to-end: plan infra with Terraform then configure with Ansible.

```

pipeline {
  agent any
  stages {
    stage('Terraform Plan') {
      steps {
        withCredentials([string(credentialsId: 'aws-access-key', variable: 'AWS_ACCESS_KEY_ID'),
                        string(credentialsId: 'aws-secret-key', variable: 'AWS_SECRET_ACCESS_KEY')]) {
          sh 'terraform init && terraform plan -out=tfplan'
        }
      }
    }
    stage('Terraform Apply (manual gate)') {
      steps {
        input message: 'Apply infrastructure changes?', ok: 'Apply'
        sh 'terraform apply -auto-approve tfplan'
      }
    }
    stage('Configure with Ansible') {
      steps {
        sh 'ansible-playbook -i inventory/prod site.yml'
      }
    }
  }
}

```